Operations Manual — Injury Outcome Dashboard

University of Michigan Injury Prevention Center

Prepared by Jaemin Jeon · August 2026

I built the Injury Outcome Dashboard as a graduate project with the IPC, and I'm writing this so that whoever takes it over after I graduate can keep it running and up to date without having to read through my R code. If all you need is the yearly data refresh, Sections 5–7 are enough on their own; the earlier sections are background and one-time setup that you'll only touch once.

Live (public) site	https://injurycenter.umich.edu/injury-outcome-dashboard/ — the IPC page, which embeds the app in an `<iframe>`
App deployment	https://ipcapp.shinyapps.io/injury-outcome-dashboard/ — shinyapps.io, account `ipcapp`
Code repository	`~/injury_outcome_dashboard` on U-M Great Lakes
Data storage	U-M Great Lakes (raw exports and the generated database both live inside the repo's `data/` folder)
Update cadence	Once a year, when the CDC publishes a new full year of data

The one thing to understand up front: the deployed app is a **static snapshot**. It reads a database (`data/injury_outcomes.sqlite`) that gets bundled *into* the deployment, and it never queries anything live. So nothing on the public site updates by itself. To put new data online I have to (1) fetch the data, (2) rebuild that database, and (3) **re-deploy**. That whole loop is Section 7.

Table of Contents

0. Quick reference

The full, careful version of the annual update is in Section 7 — this is the short form I keep on hand. The example assumes the new year is `2025`; swap in whatever the real new year is.

```
cd ~/injury_outcome_dashboard

# 1. Add the new year to the two year lists (see 7.1) -- do FIRST:
#      fetch_cdc.R : add "2025" to VALID_PERIODS
#      fetch_env.R : set VALID_YEARS <- 2019:2025

# 2. Pull the new year of CDC injury data (all types, state + county)
./fetch_cdc.sh --period 2025

# 3. Refresh NOAA environmental data (re-pulls all years, adds the new one)
./fetch_env.sh

# 4. Rebuild the database and check it locally
FORCE_DB_REBUILD=1 ./run.sh    # then open http://127.0.0.1:3838

# 5. Publish to the live site
bash deploy.sh

# 6. Save the work
git add -A && git commit -m "Add 2025 data" && git push
```

Just re-deploy what's already built:

```
cd ~/injury_outcome_dashboard
FORCE_DB_REBUILD=1 Rscript scripts/setup_db.R   # rebuild DB from disk
bash deploy.sh                                  # push to shinyapps.io
```

---

1. What the dashboard is

It's an interactive R Shiny app that maps U.S. injury mortality at the **state** and **county** level. The main features:

- **Standard choropleth** — crude death rate per 100,000, shaded in quintile bins.
- **Hotspot map** — a Gi* spatial-cluster analysis that flags statistically hot and cold regions.
- **Click-to-select** — click any state or county to fill a summary panel (rate, deaths, population, and the national average for context).
- **Environmental regression panel** — a scatter plot of the selected injury rate against annual mean temperature or total precipitation, with plain-language statistics and a built-in check that hides the numbers when the sample is too small.

What's loaded right now: six injury types (drug overdose, all suicide, all homicide, firearm deaths, firearm homicide, firearm suicide), years **2019–2024**, and a single demographic slice (`all_demographics`). I intentionally exclude the District of Columbia. See Appendix B.

One caution for anything public-facing: the demographic dropdown in the interface is scaffolding I left in for a future feature. Only `all_demographics` has data behind it today, so please don't advertise demographic breakdowns until real per-group files are added (Section 8.2).

---

2. Where everything lives

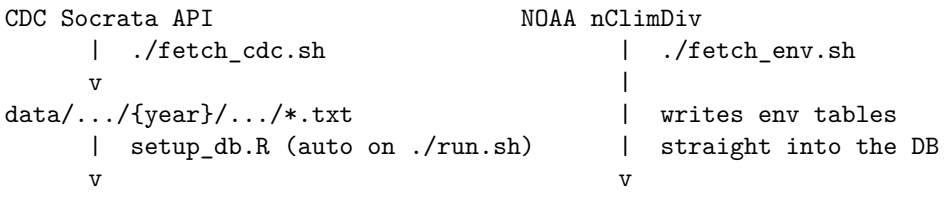
Thing	Location	Notes
Source code	`~/injury_outcome_dashboard` on Great Lakes	Keep it at this exact path — the shell scripts hard-code it (see below).
Raw injury data	`data/{level}/.../*.txt`	One tab-separated file per level, injury type, and year (full path pattern in Section 3). Written by `fetch_cdc.sh`.
Generated database	`data/injury_outcomes.sqlite`	Built from the `.txt` files by `setup_db.R`; also holds the two environmental tables. This is the file the app actually reads.
Map shapefiles	`usa_states_s.rds`, `usa_counties_s.rds`	Pre-simplified boundaries under `data/`. Static — you'll almost never touch these.
shinyapps.io account	`ipcapp`	App id and URL live in a `.dcf` file under `rsconnect/` (see Section 6).
Deploy credentials	`register_account.sh`	Git-ignored, never committed. Holds the live shinyapps.io token and secret. See Section 9.
Reference PDFs	`docs/*.pdf`	Justify the sample-size thresholds in the regression panel. Please don't delete them.

A note on the path. `run.sh`, `fetch_cdc.sh`, `fetch_env.sh`, and `deploy.sh` all call their scripts through the absolute path `~/injury_outcome_dashboard/...`, and the app itself runs from the current directory. So keep the repo at `~/injury_outcome_dashboard` and always `cd` into it before running anything. If you ever have to move it, update that path inside those four scripts.

---

3. How the dashboard is built

Everything flows in one direction. Nothing talks to a live server while the app is running — it opens the local SQLite file once at startup and reads from it. I set it up to be file-driven on purpose, so that adding data almost never means editing R.



```
data/injury_outcomes.sqlite  <-------------- +
      |  read once at startup (global.R)
      v
Shiny app  (app.R -> global.R + ui.R + server.R)
      |  ./deploy.sh  (bundles the code + the .sqlite)
      v
ipcapp.shinyapps.io/injury-outcome-dashboard
      |  embedded via an iframe
      v
injurycenter.umich.edu/injury-outcome-dashboard   (public site)
```

There are two data sources, and they land in two different places:

1. **Injury data** — `fetch_cdc.sh` pulls from the CDC `data.cdc.gov` Socrata API and writes `.txt` files into the `data/` folder tree. `setup_db.R` then compiles those files into the SQLite database. The folder path *is* the metadata: the injury type, year, and demographic are read straight off the directory names, so the dropdowns populate themselves from whatever files exist.
2. **Environmental data** — `fetch_env.sh` pulls NOAA nClimDiv climate data and writes it **directly into the SQLite database** as the `env_by_state` / `env_by_county` tables. It doesn't create `.txt` files and doesn't need a database rebuild.

The map shapefiles (`usa_states_s.rds`, `usa_counties_s.rds`) are the polygons the maps are drawn on. They're pre-generated and effectively permanent — state and county boundaries don't change year to year — so the annual update never touches them.

---

4. One-time setup

Do this once per maintainer, or once on a fresh Great Lakes account. If the app already runs for you, skip ahead to Section 7.

4.1 Get onto Great Lakes and find the code

Log into U-M Great Lakes (SSH, VS Code Remote-SSH, or Open OnDemand). The repo should be at `~/injury_outcome_dashboard`. If it isn't there, clone it from wherever IPC keeps the git remote into that exact path (see the path note in Section 2).

4.2 Load the R environment

Everything runs on one R module. The wrapper scripts load it for you, but if you run R by hand, load it first:

```
module load Rgeospatial/4.5.1-2025-10-07
```

4.3 Install the extra R packages (first time only)

The `Rgeospatial` module provides most of the dependencies; a few more go into your personal R library. This can take 10+ minutes the first time.

```
cd ~/injury_outcome_dashboard
bash install_packages.sh
```

4.4 Register shinyapps.io credentials (only needed to deploy)

Deployment publishes to the shared `ipcapp` shinyapps.io account. You register its token and secret once per machine:

```
bash register_account.sh
```

Don't commit `register_account.sh` — it contains a live secret and is already in `.gitignore`. If you don't have the credentials, get them from whoever owns the `ipcapp` account, or generate a fresh token/secret while logged into that account at https://www.shinyapps.io/admin/#/tokens and paste them into your own copy of the script. More on this in Section 9.

---

5. Running and viewing it locally

```
cd ~/injury_outcome_dashboard
./run.sh
```

On every start, `run.sh` runs `global.R`, which checks whether any `.txt` file under `data/` is newer than the database and, if so, rebuilds the database automatically before the app boots. You'll see either `Data changes detected -- rebuilding SQLite` or `SQLite is up to date -- skipping rebuild`, and then:

```
Listening on http://127.0.0.1:3838
```

To open it:

- **VS Code Remote-SSH:** click the "Open in Browser" popup for port 3838, or use the **Ports** tab at the bottom.
- **Open OnDemand or a desktop session:** browse to `http://127.0.0.1:3838`.
- **Plain SSH:** forward the port when you connect (`ssh -L 3838:localhost:3838 you@greatlakes.arc-ts.umich.edu`), then open `http://localhost:3838` on your laptop.

Stop the app with **Ctrl+C**.

If you ever *delete* a data file, or you just suspect the database is stale, force a clean rebuild — the automatic check only notices new or modified files, not deletions:

```
FORCE_DB_REBUILD=1 ./run.sh
```

---

6. Deploying to the live site

Deploying bundles the app code **and the current `data/injury_outcomes.sqlite`** and pushes them to shinyapps.io.

```
cd ~/injury_outcome_dashboard

# 1. Make sure the bundled database reflects the current data files
FORCE_DB_REBUILD=1 Rscript scripts/setup_db.R   # or run ./run.sh once

# 2. Deploy
bash deploy.sh
```

Always rebuild the database before deploying. `deploy.sh` uploads whatever `.sqlite` is on disk at that moment, so if you fetched new data but never rebuilt, you'll publish the old numbers. When it finishes (a few minutes), check the result at https://ipcapp.shinyapps.io/injury-outcome-dashboard/. The IPC page embeds that URL, so it picks up the change automatically.

---

7. The annual update runbook

This is the recurring yearly job — the main reason I wrote this manual. When the CDC publishes a new complete year of the *Mapping Injury, Overdose, and Violence* data, this is exactly what I do to get it onto the public site. The steps below add year **2025** as the example; substitute the real new year throughout.

7.0 When to run it

The CDC finalizes a calendar year's mortality data roughly a year later (so 2025 data tends to appear during 2026). You don't have to know the exact date — Step 7.2 simply reports "no data returned" if the year isn't published yet, and you try again in a month or two.

7.1 Extend the two hard-coded year lists — do this first

This is the step I'm most worried about you missing, because it fails quietly rather than with an obvious error. Two scripts have the valid year range hard-coded, and each one will reject or discard any year outside that range:

File	What to change	Why it matters
`scripts/fetch_cdc.R`	Add the new year to `VALID_PERIODS` (e.g. `..., "2024", "2025"`)	Without it, `./fetch_cdc.sh --period 2025` stops with `--period must be one of...` and fetches nothing.

File	What to change	Why it matters
`scripts/fetch_env.R`	Change `VALID_YEARS <- 2019:2024` to `2019:2025`	Without it, the new year's climate data downloads but is then filtered out, so the scatter panel has no environmental data for the new year.

Edit both, save, and you're ready to fetch.

7.2 Fetch the new year of CDC injury data

```
cd ~/injury_outcome_dashboard
./fetch_cdc.sh --period 2025
```

This pulls all six injury types, at both state and county level, for that year, and writes them under `data/{level}/{injury_type}/2025/all_demographics/`. Watch the output:

- Lines like `wrote 51 rows -> data/state/drug_overdose/2025/...` mean it worked.
- `no state data returned` / `no county data returned` for *every* type means the CDC hasn't published that year yet — stop here and come back later.

(To refresh an existing year instead of adding a new one, run the same command with that year and it overwrites the files in place. To re-pull everything, run `./fetch_cdc.sh` with no flags.)

7.3 Refresh the environmental data

```
./fetch_env.sh
```

NOAA updates monthly and this script always re-downloads the full history, so running it refreshes every year and — because you extended `VALID_YEARS` in 7.1 — adds the new one. It writes straight into the database and takes about 30–60 seconds. One thing to know: NOAA nClimDiv covers the 48 continental states only, so Alaska and Hawaii never have temperature or precipitation points in the scatter panel. That's expected, not a bug.

7.4 Rebuild the database and check it locally

```
FORCE_DB_REBUILD=1 ./run.sh
```

Open `http://127.0.0.1:3838` and confirm:

- The **Period** dropdown now lists the new year.
- Selecting the new year repaints the map at both state and county level.
- The **Hotspot** map renders for the new year.
- The **scatter panel** shows points for the new year (both temperature and precipitation are selectable).

Stop the app with **Ctrl+C**.

7.5 Deploy to the live site

```
bash deploy.sh
```

Wait for it to finish, then verify at https://ipcapp.shinyapps.io/injury-outcome-dashboard/. The IPC website page updates on its own, since it's just an iframe of that URL.

7.6 Commit and push

Save the new data files **and** the two edited scripts, so the next person inherits a working copy:

```
git add -A
git commit -m "Add 2025 data (CDC injury + NOAA environmental)"
git push
```

7.7 Annual update checklist

- `scripts/fetch_cdc.R` — new year added to `VALID_PERIODS`
- `scripts/fetch_env.R` — `VALID_YEARS` end year bumped
- `./fetch_cdc.sh --period <YEAR>` ran and wrote files (not "no data returned")
- `./fetch_env.sh` ran successfully
- `FORCE_DB_REBUILD=1 ./run.sh` — new year checked on both map types and the scatter
- `bash deploy.sh` — live URL shows the new year
- `git commit` and `git push`

---

8. Other maintenance tasks

8.1 Add a new injury type

Adding data never means editing the app itself — but the *fetcher* only knows the six injury types listed in its `INTENT_MAP`. To add a CDC injury type that already exists in the Socrata dataset, add it to `INTENT_MAP` in `scripts/fetch_cdc.R` (`<API intent value> = "<folder_name>"`) and run `./fetch_cdc.sh --injury <intent>`.

To add data from a different source (say, a manual CDC WONDER export), skip the fetcher and just drop the file into the right folder by hand:

```
data/{level}/{injury_type}/{period}/{demographic}/<file>.txt
```

Use lowercase snake_case folder names — those names become the dropdown labels automatically (`firearm_suicide` turns into "Firearm Suicide"). The file has to be a tab-separated, CDC-WONDER-style export with the standard columns. Then rebuild (`FORCE_DB_REBUILD=1 ./run.sh`) and deploy.

8.2 Add a new demographic breakdown

Same folder rule — the fourth path level is the demographic. To add, for example, male overdose data, place files under `.../drug_overdose/2024/male/`, and the **Demographics** dropdown fills in on its own. (The CDC Socrata feed the fetcher uses returns all-demographics totals only, so for now demographic files have to come from another export source.)

8.3 Remove a dataset

Delete the `.txt` file(s), then rebuild with a forced rebuild — the automatic timestamp check can't detect deletions on its own:

```
FORCE_DB_REBUILD=1 ./run.sh
```

8.4 Update the map shapefiles (rare)

`usa_states_s.rds` and `usa_counties_s.rds` are pre-simplified boundary files and only need replacing if county lines change or you want different simplification. This is outside the yearly cycle and not part of routine maintenance.

---

9. Long-term handoff

Right now two things the dashboard depends on are tied to my personal U-M account, and both need to move to IPC before that account expires. They are separate systems, so I treat them separately:

- **Great Lakes storage** (Section 9.1) — where the code and data live, and where the app is rebuilt and deployed from.
- **The live shinyapps.io account** (Section 9.2) — the outside service that actually hosts the running app and serves the public URL.

The key thing to understand is that these are independent. The live dashboard runs on shinyapps.io, *not* on Great Lakes, so it keeps serving the public page even if the Great Lakes files go away. Great Lakes only matters when someone needs to rebuild the data or push an update. Section 9.3 covers the git repository and Section 9.4 the plan limits.

9.1 Move the files to IPC-owned Great Lakes / Turbo storage

Everything the app needs is one folder — `~/injury_outcome_dashboard` — with no external database or service behind it, so moving it is just moving that folder to storage IPC controls.

For IPC: request a research storage allocation from U-M ARC. Turbo is the usual choice for a long-lived research dataset, and Great Lakes offers space as well; either works. It's a standard, low-cost (often free for research) request at https://arc.umich.edu, and it can be owned by a lab or PI rather than a student, so it doesn't expire when a student leaves.

Once the allocation exists, the transfer is a single copy of the whole folder, for example:

```
cp -r ~/injury_outcome_dashboard /nfs/turbo/<ipc-allocation>/
```

That brings the raw `.txt` files, the generated `.sqlite`, and the shapefiles along with it. From then on, run and deploy from the new location.

9.2 Move the live app to an IPC-owned shinyapps.io account

The app is published to shinyapps.io, a hosting service run by Posit. Today it deploys to an account tied to me; to make it IPC's, someone at IPC needs to own the account and hand the maintainer a deploy token. **No R or coding knowledge is needed for the IPC side** — it's all done in a web browser. The maintainer then makes a one-line change to a script and re-publishes.

Part A — For IPC (web browser only, no coding):

1. Go to https://www.shinyapps.io and sign in. If IPC doesn't have an account yet, click **Sign Up** and register with an **IPC-owned email** — a shared mailbox or a permanent staff member's address, *not* a student's. Signing in with Google or GitHub is fine as long as that login belongs to IPC.
2. The first time, it asks you to choose an **account name**. This name becomes part of the public web address (`https://<account-name>.shinyapps.io/...`), so pick something clean like `umichipc`. Write down exactly what you chose — the maintainer needs it.
3. Click your account name in the top-right corner and choose **Tokens** (or go straight to https://www.shinyapps.io/admin/#/tokens).
4. Click **+ Add Token**. A new token row appears.
5. Click **Show** on that row, then **Show secret**. You'll see a line of text like this (the long values will differ):

   ```
   rsconnect::setAccountInfo(
     name   = '...',
     token  = '...',
     secret = '...')
   ```

6. Copy that entire block and send it to the maintainer. Treat it like a password — the `secret` lets anyone publish to the account — so keep it to a trusted channel. A U-M email between IPC staff and the maintainer is fine; just don't post it anywhere public or commit it to git.

That is the whole IPC side. Once the maintainer has that block, the rest is on them.

Part B — For the maintainer (the person with the code):

1. In the repo root there is a small file, `register_account.sh`. It contains exactly one meaningful command — a call to `setAccountInfo` with a `name`, a `token`, and a `secret`:

   ```
   Rscript -e "rsconnect::setAccountInfo(
     name   = 'ACCOUNT_NAME',
     token  = 'PASTE_TOKEN_HERE',
     secret = 'PASTE_SECRET_HERE')"
   ```

   (In the real file this is all on one line — that's fine, it works either way.)

2. Replace the three values with the ones IPC sent, and save the file. Never commit it; it holds a live secret and is already in `.gitignore`.
3. Register the new account and re-publish:

   ```
   cd ~/injury_outcome_dashboard
   bash register_account.sh
   bash deploy.sh
   ```

4. **If IPC chose a new account name**, the public address changes to `https://<new-name>.shinyapps.io/..` (same app path, new account). Tell whoever maintains the website so they update the `<iframe>` on the IPC page to point at it. **If the account name is the same as before (`ipcapp`)**, the address doesn't change and the website needs no edit.
5. Finally, return to the shinyapps.io **Tokens** page and delete the old token tied to the previous maintainer's setup, so that old copy can no longer publish.

9.3 The git repository

Make sure IPC controls the git remote — for example a repo in a U-M GitHub organization — so the code isn't tied to a personal account. Everything except `register_account.sh` (which is git-ignored) is safe to commit.

9.4 shinyapps.io plan limits

The free tier caps how many active hours the app can run each month and how many apps one account can host. If the public page gets steady traffic, IPC may need a paid plan so the dashboard doesn't go to sleep or hit its monthly limit. This is set on the same shinyapps.io account, under its billing/plan settings.

---

10. Troubleshooting

A new year, type, or demographic doesn't show up after I added data. Rebuild with `FORCE_DB_REBUILD=1 ./run.sh`, then read the boot log for a line like `injury_by_state: N rows written from M file(s)`. If `M` is lower than you expect, a file is at the wrong folder depth — the path has to be exactly `data/{level}/{injury_type}/{period}/{demographic}/<file>.txt`.

`./fetch_cdc.sh --period 2025` stops with `--period must be one of....` You skipped Step 7.1 — add the year to `VALID_PERIODS` in `scripts/fetch_cdc.R`.

The new year shows on the map but the scatter panel has no points for it. The environmental data wasn't extended. Bump `VALID_YEARS` in `scripts/fetch_env.R` (Step 7.1), re-run `./fetch_env.sh`, rebuild, and redeploy.

"No data for the selected Injury Type / Period / Demographic combination." That combination has no file behind it. Pick another one, or add the file.

The scatter panel says "Not enough data." The regression needs at least 10 valid points, and 30 or more for the full output. For a rare injury at the state level this can legitimately be too few — it's the app guarding against misleading statistics, not a bug.

`fetch_env.sh` fails to download. NOAA's server was unreachable or changed a filename. Re-run later; the script re-discovers the latest filename each time.

Deploy succeeded but the live site shows old numbers. You deployed without rebuilding the database. Run `FORCE_DB_REBUILD=1 Rscript scripts/setup_db.R`, then `bash deploy.sh` again.

`module: command not found`. The shell doesn't have Lmod initialized. Log out and back into Great Lakes, or run `source /usr/share/lmod/lmod/init/bash`.

`Permission denied` when running a script. Make them executable: `chmod +x run.sh fetch_cdc.sh fetch_env.sh deploy.sh install_packages.sh register_account.sh`.

Port 3838 is already in use. An old R session is still running. Find it with `ps -ef | grep run_app` and stop it with `kill <PID>`.

Appendix A — File and directory reference

```
injury_outcome_dashboard/
  app.R          sources global.R, ui.R, server.R
  global.R       boot: rebuild-if-stale, open DB, load tables/shapefiles
  ui.R           layout and controls
  server.R       filtering, maps, hotspot, summary, scatter

  scripts/
    setup_db.R   builds injury_outcomes.sqlite from data/*.txt
    fetch_cdc.R  CDC API -> data/*.txt   (edit VALID_PERIODS/year)
    fetch_env.R  NOAA -> env tables      (edit VALID_YEARS/year)
    run_app.R    runApp(".", port = 3838)

  run.sh                 run the app (auto-rebuilds DB if stale)
  fetch_cdc.sh           wrapper for fetch_cdc.R (passes flags)
  fetch_env.sh           wrapper for fetch_env.R
  deploy.sh              deploy to shinyapps.io
  install_packages.sh    first-time R package install
  register_account.sh    shinyapps.io creds (git-ignored; never commit)

  data/
    injury_outcomes.sqlite   generated; also holds env_by_state/county
    usa_states_s.rds         state map polygons (static)
    usa_counties_s.rds       county map polygons (static)
    state/  {injury}/{year}/{demographic}/state_*.txt
    county/ {injury}/{year}/{demographic}/county_*.txt

  docs/
    OPERATIONS_MANUAL.md   this file
    DATA_FETCH.md          data-source reference (endpoints, formats)
```

`*.pdf` `regression sample-size references (keep)`

Appendix B — Injury types and data sources

Injury types (folder name and the matching CDC API `intent` value):

Folder / dropdown	CDC API intent	Meaning
`drug_overdose`	`Drug_OD`	Unintentional or undetermined drug overdose deaths
`all_suicide`	`All_Suicide`	All-mechanism suicide deaths
`all_homicide`	`All_Homicide`	All-mechanism homicide deaths
`firearm_deaths`	`FA_Deaths`	All firearm deaths, any intent
`firearm_homicide`	`FA_Homicide`	Firearm homicide deaths
`firearm_suicide`	`FA_Suicide`	Firearm suicide deaths

Data sources:

- **Injury data** — CDC *Mapping Injury, Overdose, and Violence*, via the `data.cdc.gov` Socrata API (state `fpsi-y8tj`, county `psx4-wq38`). No API key. Coverage 2019 to present. Counts of 1–9 are CDC-suppressed and stored as an "Unreliable" sentinel value.
- **Environmental data** — NOAA NCEI **nClimDiv** county temperature (`tmpccy`, in degrees F) and precipitation (`pcpncy`, in inches). No API key. 48 continental states only.

Full endpoint details, file formats, and the note on why the CDC WONDER API was ruled out are in `docs/DATA_FETCH.md`.

I originally developed this dashboard as part of my graduate work at the University of Michigan. Please keep this manual updated as the build, deploy, or update process changes. — Jaemin Jeon